



Math Masters Library Audit Report

Version 1.0

meharab.github.io

June 30, 2026

Math Masters Library Audit Report

Meharab

June 30, 2026

Prepared by: CodeBreakers

Lead Auditors: - Meharab

Assisting Auditors: - None

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] The `MathMasters::mulWadUp` function gives incorrect result for some values of x and y
 - * [H-2] `MathMasters::mulWadUp` function does not revert as expected
 - * [H-3] Use of Decimals in `MathMasters::sqrt()` instead of Hexadecimals representation results in nonequivalence leading to loss of precision.

– Low

- ★ [L-1] Version compatibility issue prevents use of library for contracts using version 0.8.3 of Solidity
- ★ [L-2] In `MathMasters::mulWad` and `MathMasters::mulWadUp` functions the revert reason is empty
- ★ [L-3] In `MathMasters::mulWad` and `MathMasters::mulWadUp` functions the revert reason is empty

Protocol Summary

It is a Math Library.

Disclaimer

The CodeBreakers team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond the following commit hash:

1 c7643faa1a188a51b2167b68250816f90a9668c6

Scope

```
src
└─ MathMasters.sol
```

Roles

- Developer: It is a public library

Executive Summary

Spend 96 hours and found 3 high, 0 medium, 3 low, 0 informational, and 0 gas in total 6 issues.

Issues found

Severity	Number of issues found
High	3
Medium	0
Low	3
Info	0
Gas	0
Total	6

Findings

High

[H-1] The MathMasters::mulWadUp function gives incorrect result for some values of x and y

Summary

The MathMasters::mulWadUp function should calculate the expression $(x * y) / 1e18$ and round the result up. The function correctly checks if there is a remainder and adds 1 if it is necessary. But by some values for x and y the function increments x with 1 which leads to incorrect calculation and incorrect final result of z.

Vulnerability Details

The function MathMasters::mulWadUp accepts two input parameters (uint256 x and uint256 y) and calculates the expression $x * y / 1e18$ and rounds the result up.

There is an if statement in the function that increments the x value with 1. Maybe, the reason for doing that is the result to be rounded up, but this is incorrect.

```

/// @dev Equivalent to `(x * y) / WAD` rounded up.
function mulWadUp(uint256 x, uint256 y) internal pure returns (uint256 z) {
    /// @solidity memory-safe-assembly
    assembly {
        // Equivalent to `require(y == 0 || x <= type(uint256).max / y)`.
        if mul(y, gt(x, or(div(not(0), y), x))) {
            mstore(0x40, 0xbac65e5b) // `MathMasters__MulWadFailed()`.
            revert(0x1c, 0x04)
        }
    }
    @> if iszero(sub(div(add(z, x), y), 1)) { x := add(x, 1) }
    z := add(iszero(iszero(mod(mul(x, y), WAD))), div(mul(x, y), WAD))
}

```

Let's take a look to the if statement that increments the value of x: `iszero(sub(div(add(z, x), y), 1))`. - `add(z, x)`: This adds z and x. But z is supposed to be the result from the MathMasters::mulWadUp function, it is not initialized and therefore its value is 0. So, this addition is useless. - `div(add(z, x), y)`: This divides the result of addition that is x by the value of y. - `sub(div(add(z, x), y), 1)`: This subtracts the result of division by 1. - `iszero(sub(div(add(z, x), y), 1))`: This checks if the result after subtraction is 0. If it is 0, the if statement will be true and the x value will be incremented by 1.

When this result can be 0? - If the values of x and y are equal. - If the value of x is slightly greater than the value of y but not enough to make the division result reach 2 when rounded down to the nearest integer.

In these cases the value of x will be incremented by 1. But this incrementation leads to incorrect final result. In the last line of the function is assigned the final result to the variable z : $z := \text{add}(\text{iszero}(\text{iszero}(\text{mod}(\text{mul}(x, y), \text{WAD}))), \text{div}(\text{mul}(x, y), \text{WAD}))$. Also, there is a check if the result of the expression $(x * y) / \text{WAD}$ has a remainder. If there is a remainder, 1 is added to round up the result.

Impact

Let's consider the following scenario:

The values of x and y are both equal to $3e18$. The following test function `testMulWadUpCalculation` calculates the expected solution and compares it to the solution retrieved from the `MathMasters::mulWadUp` function. You can add this test function in the `MathMasters.t.sol` file and execute it with the Foundry command: `forge test --match-test "testMulWadUpCalculation"`

```
function testMulWadUpCalculation() public {
    uint256 solution = (3e18 * 3e18) / 1e18;
    if (solution * 1e18 < 3e18 * 3e18) {
        solution += 1;
    }

    assertEq(MathMasters.mulWadUp(3e18, 3e18), solution);
}
```

Test fails:

[FAIL. Reason: assertion failed] `testMulWadUpCalculation()` (gas: 21368)

Logs:

```
Error: a == b not satisfied [uint]
Left: 9000000000000000003
Right: 9000000000000000000
```

The result from the `MathMasters::mulWadUp` function is 9000000000000000003 and the expected result is 9000000000000000000. This difference is because the incorrect incrementation of the value of x .

The following test in `Hałmos` found also the case by which the value of x is slightly greater than the value of y but not enough to make the division result reach 2 when rounded down to the nearest integer. `Hałmos` is a great tool for formal verification. To execute this test you should have `Hałmos`

installed and import the SymTest from `lib/halmos-cheatcodes/src/SymTest.sol`. Then you can use the command to execute the test function: `halmos --function check_check_MulWadUp --solver-timeout-assertion 0`.

```
function check_MulWadUp(uint128 x, uint128 y) public {
    unchecked {
        if (x != 0 && (x * y) / x != y) return;
    }

    uint128 solution = x * y / 1e18;
    if (solution * 1e18 < x * y) {
        solution += 1;
    }

    uint256 mathSolution = MathMasters.mulWadUp(x,y);
    assertEq(solution, mathSolution);
}
```

The input arguments for the functions are of type uint128, because with the type uint256 the result was Killed. So, the HalmoS found the counterexample: `x = 20901944742440337407` and `y = 15364007485707028177`. I stopped HalmoS after the first found example because it took about 30 minutes. After that, I decided to compare the results from `MathMasters::mulWadUp` function and `Solady::mulWadUp` function. The following test demonstrates this, again with the help of HalmoS:

```
function testCheck__MulWadUpEquivalence(uint256 x, uint256 y) public {

    uint256 mathMasters = MathMasters.mulWadUp(x, y);
    //mulWadUp2 is the name of `Solady::mulWadUp` function in the
    ↪ `MathMasters.sol` file
    uint256 solady = MathMasters.mulWadUp2(x, y);

    assertEq(mathMasters, solady);
}
```

This time the type of the input arguments is not changed and the result is:

```
p_x_uint256 =
↪ 0x000000000000000000000000000000000000000000000000000000000000000013806bd089fb6d2f4ec4bba7f87c865be
↪ (414754122524714488867341405485055632830)
```

```
p_uint256 =  
↳ 0x00000000000000000000000000000000ccf76b10a93fec70b20605325b922003  
↳ (272447180002039376997384714789787410435)  
[FAIL] testCheck__MulWadUpEquivalence(uint256,uint256) (paths: 9, time:  
↳ 1714.20s, bounds: [])
```

All tests show that the function `MathMasters::mulWadUp` calculates the expression $(x * y) / 1e18$ incorrectly due to the unnecessary increment of `x`.

Tools Used

VS Code, Foundry, Halmos

Recommendations

Remove the `if` statement that increments the value of `x`.

```
function mulWadUp(uint256 x, uint256 y) internal pure returns (uint256 z) {
    /// @solidity memory-safe-assembly
    assembly {
        // Equivalent to `require(y == 0 || x <= type(uint256).max / y)`.
        if mul(y, gt(x, or(div(not(0), y), x))) {
            mstore(0x40, 0xbac65e5b) // `MathMasters__MulWadFailed()`.
            revert(0x1c, 0x04)
        }
        if iszero(sub(div(add(z, x), y), 1)) { x := add(x, 1) }
        z := add(iszero(iszero(mod(mul(x, y), WAD))), div(mul(x, y), WAD))
    }
}
```

[H-2] MathMasters: :mulWadUp function does not revert as expected

Relevant GitHub Links

<https://github.com/Cyfrin/2024-01-math-master/blob/84c149baf09c1558d7ba3493c7c4e68d83e7b3aa/src/MathMas>

Summary

The `MathMasters::mulWadUp` function should revert if the input parameters `uint256 x` and `uint256 y` do not satisfy the condition: `y == 0 || x <= type(uint256).max / y`. But the function does not revert as expected.

Vulnerability Details

The function `MathMasters::mulWadUp` accepts two input parameters (`uint256 x` and `uint256 y`) and calculates the expression $x * y / 1e18$ and rounds the result up. The function has check for input values which lead to overflow. In the comment of the function is written that it is required y to be 0 or $x \leq \text{type}(\text{uint256}).\text{max} / y$. But the if statement that should check for this conditions does not check correctly:

```

/// @dev Equivalent to `(x * y) / WAD` rounded up.
function mulWadUp(uint256 x, uint256 y) internal pure returns (uint256 z) {
    /// @solidity memory-safe-assembly
    assembly {
        // Equivalent to `require(y == 0 || x <= type(uint256).max / y)`.
        @> if mul(y, gt(x, or(div(not(0), y), x))) {
            mstore(0x40, 0xbac65e5b) // `MathMasters__MulWadFailed()`.
            revert(0x1c, 0x04)
        }
        if iszero(sub(div(add(z, x), y), 1)) { x := add(x, 1) }
        z := add(iszero(iszero(mod(mul(x, y), WAD))), div(mul(x, y), WAD))
    }
}

```

Let's consider the if statement: `mul(y, gt(x, or(div(not(0), y), x)))`: `- not(0)`: This computes the bitwise not of `0`, which results in a value where all bits are set to `1`. In the context of a `uint256`, this is equivalent to the maximum possible value for a `uint256`, which is $2^{256} - 1$. `- div(not(0), y)`: This calculates the maximum value that can be safely multiplied by `y` without causing an overflow. `- or(div(not(0), y), x)`: This performs a bitwise or between the result of `div(not(0), y)` and `x`. However, this doesn't make sense in the context of an overflow check, as the or operation here would not serve a purpose in preventing overflow. `- gt(x, or(div(not(0), y), x))`: This will return `1` if `x` is greater than the result of a bitwise or operation and `0` otherwise. `- mul(y, gt(x, or(div(not(0), y), x)))`: The final operation will multiply `y` with the result of `gt` operation.

The described overflow check bypasses values for `x` and `y` which does not meet the requirement: $x \leq \text{type}(\text{uint256}).\text{max} / y$ and result in an overflow.

Impact

Let's consider the following scenario: The value of the input parameter `uint256 x` is `2` and the value of the second input parameter `uint256 y` is `5789604461865809771178549250434395392663499233282` which is 2^{255} . The requirement: $x \leq \text{type}(\text{uint256}).\text{max} / y$ is not satisfied because $\text{type}(\text{uint256}).\text{max} / y$ is `1` and `x` is `2`. Therefore, the if statement in `MathMasters::mulWadUp` function should be true and the function should revert. But the if statement returns false and the function continue with the calculations.

Let's consider again the `if` statement but this time with values for `x` and `y`: `not(0)` is equivalent to $2^{256} - 1$. `div(not(0), y)` divides the maximum `uint256` value by `y`. If `y` is 2^{255} , this division results in 2. `gt(x, or(div(not(0), y), x))` checks if `x` is greater than the result of the division. Since `x` is 2, it is not greater than 2, so the `gt` function returns 0. `mul(y, gt(x, or(div(not(0), y), x)))` multiplies `y` by the result of the `gt` function. Since `gt` returns 0, the multiplication result is 0, and the condition inside the `if` statement is `false`.

In that way the function will not revert as expected.

The following test function `testMulWadUpFuzzOverflow` in Foundry found the described scenario. You can add the test function to the file `MathMasters.t.sol` and execute it with the command: `forge test --match-test "testMulWadUpFuzzOverflow"`

```
function testMulWadUpFuzzOverflow(uint256 x, uint256 y) public {  
    // Precondition: x * y > uint256 max  
    // After reviewing the code, I know it will be enough x to be a small  
    ↪ number greater than one, therefore x is limited to be lower than  
    ↪ 10  
    vm.assume(x > 1 && x < 10);  
    vm.assume(y > type(uint256).max / x);  
  
    vm.expectRevert();  
    MathMasters.mulWadUp(x, y);  
}
```

At the end the result of the function `MathMasters::mulWadUp` with the provided arguments will be incorrect due to the overflow.

Tools Used

VS Code, Foundry

Recommendations

Change the condition in the `if` statement in the `MathMasters::mulWadUp` function to ensure that the values of `x` and `y` satisfy the condition: `y == 0 || x <= type(uint256).max / y`.

```
function mulWadUp(uint256 x, uint256 y) internal pure returns (uint256 z) {  
    /// @solidity memory-safe-assembly  
    assembly {  
        // Equivalent to `require(y == 0 || x <= type(uint256).max / y)`.  
+        if mul(y, gt(x, div(not(0), y))) {
```

```
- if mul(y, gt(x, or(div(not(0), y), x))) {
    mstore(0x40, 0xbac65e5b) // `MathMasters__MulWadFailed()`.
    revert(0x1c, 0x04)
}
if iszero(sub(div(add(z, x), y), 1)) { x := add(x, 1) }
z := add(iszero(iszero(mod(mul(x, y), WAD))), div(mul(x, y), WAD))
}
```

[H-3] Use of Decimals in MathMasters : : sqrt () instead of Hexadecimals representation results in nonequivalence leading to loss of precision.

Relevant GitHub Links

<https://github.com/Cyfrin/2024-01-math-master/blob/84c149baf09c1558d7ba3493c7c4e68d83e7b3aa/src/MathMas>

Summary

Use of Decimals in `MathMasters::sqrt()` instead of Hexadecimals representation results in nonequivalence leading to loss of precision.

Vulnerability Details

The MathMasters implementation is based on the equivalent soladay version. However, the MathMaster implementation uses decimals to represent the constants, while the soladay implementation uses hexadecimals. In Ethereum's Solidity programming language, hexadecimal notation (base-16) is commonly used for specifying literals and values, especially when working with low-level constructs like assembly. Hexadecimal notation is more aligned with the way data is represented in the Ethereum Virtual Machine (EVM). While you can use decimal literals in high-level Solidity code, when writing assembly code in Solidity, it's generally expected to use hexadecimal notation for representing values and memory locations. In short, the EVM is expecting to receive hexadecimal values and MathMaster sqrt function is submitting decimal values instead. This results in a loss of precision in the MathMasters implementation.

PoC

- Step 1: As a reference implementation refer to solady sqrt: <https://github.com/Vectorized/solady/blob/8919f61>
- Step 2: Notice that that bottom half of both sqrt implementations (MathMasters and Solady) are identical. Let's place the identical code in a helper function.

```
function _identicalCodeSqrt(uint256 x, uint256 z) public pure returns
    ↪ (uint256 ret) {
        assembly {
```

```

        z := shr(1, add(z, div(x, z)))
        z := shr(1, add(z, div(x, z)))
        z := shr(1, add(z, div(x, z)))
        z := shr(1, add(z, div(x, z)))
        z := shr(1, add(z, div(x, z)))
        z := shr(1, add(z, div(x, z)))
        z := shr(1, add(z, div(x, z)))

        ret := sub(z, lt(div(x, z), z))
    }
}

```

- Step 3: We can then create versions of the two sqrt functions with calls to the identical code.

*// The Solady sqrt function with the bottom half replaced with a call to the
 ↪ helper function for the identical code.*

```

function sharedSoladySqrt(uint256 x) public pure returns (uint256 z) {
    assembly {
        z := 181

        let r := shl(7, lt(0xffffffffffffffffffffffffffffffff, x))
        r := or(r, shl(6, lt(0xffffffffffffffff, shr(r, x))))
        r := or(r, shl(5, lt(0xffffffff, shr(r, x))))
        r := or(r, shl(4, lt(0xffff, shr(r, x))))
        z := shl(shr(1, r), z)

        z := shr(18, mul(z, add(shr(r, x), 65536)))
    }
    z = _identicalCodeSqrt(x, z);
}

```

*// The MathMaster sqrt function with the bottom half replaced with a call to
 ↪ the helper function for the identical code.*

```

function sharedMathMastersSqrt(uint256 x) internal pure returns (uint256
    ↪ z) {
    /// @solidity memory-safe-assembly
    assembly {
        z := 181

        let r := shl(7, lt(87112285931760246646623899502532662132735, x))
        r := or(r, shl(6, lt(4722366482869645213695, shr(r, x))))
        r := or(r, shl(5, lt(1099511627775, shr(r, x))))
        // Correct: 16777215 0xfffff
        r := or(r, shl(4, lt(16777002, shr(r, x))))
    }
}

```

```

        z := shl(shr(1, r), z)

        z := shr(18, mul(z, add(shr(r, x), 65536))) // A `mul()` is saved
        ↪ from starting `z` at 181.
    }
    z = _identicalCodeSqrt(x, z);
}

```

- Step 4: We can then conduct a fuzz test to confirm that the two modified sqrt functions along with the helper function are still working correctly.

```

// Fuzz test to check that output for both sqrt functions with a shared
↪ helper function containing duplicate code appears to be correct.
function test_fuzz_sharedSqrtFunctions(uint32 fuzzedSolution, uint32
↪ fuzzedRandomNum) public {
    // ARRANGE: specify input conditions
    uint256 solution = fuzzedSolution;
    vm.assume(solution > 0);
    uint256 randomNum = fuzzedRandomNum;
    uint256 squaredPlusRemainder = solution * solution + (randomNum %
↪ solution);

    // ACT: call target contracts
    uint256 mathMastersOuput =
↪ Base_Test.sharedMathMastersSqrt(squaredPlusRemainder);
    uint256 soladyOutput =
↪ Base_Test.sharedSoladySqrt(squaredPlusRemainder);

    // ASSERT: check output states
    assertEq(solution, mathMastersOuput);
    assertEq(solution, soladyOutput);
}

```

- Step 5: Once we've verified that the adjusted square root functions are functioning as intended, we can disregard the duplicated code and focus on testing the essential differences in the remaining code for equivalence. To begin, let's create the two square root functions, each containing only their core code.

```

// The core Solady function, without the identical code.
function coreSoladySqrt(uint256 x) public pure returns (uint256 z) {
    assembly {
        z := 181

        let r := shl(7, lt(0xffffffffffffffffffffffffffffffff, x))
    }
}

```

```

    r := or(r, shl(6, lt(0xffffffffffffffff, shr(r, x))))
    r := or(r, shl(5, lt(0xffffffff, shr(r, x))))
    r := or(r, shl(4, lt(0xffff, shr(r, x))))
    z := shl(shr(1, r), z)

    z := shr(18, mul(z, add(shr(r, x), 65536)))
  }
}

// The core MathMaster function, without the identical code.
function coreMathMasterSqrt(uint256 x) internal pure returns (uint256 z)
↪ {
  /// @solidity memory-safe-assembly
  assembly {
    z := 181

    // This segment is to get a reasonable initial estimate for the
    ↪ Babylonian method. With a bad
    // start, the correct # of bits increases ~linearly each
    ↪ iteration instead of ~quadratically.
    let r := shl(7, lt(87112285931760246646623899502532662132735, x))
    r := or(r, shl(6, lt(4722366482869645213695, shr(r, x))))
    r := or(r, shl(5, lt(1099511627775, shr(r, x))))
    // Correct: 16777215 0xffffffff
    r := or(r, shl(4, lt(16777002, shr(r, x))))
    z := shl(shr(1, r), z)

    // There is no overflow risk here since `y < 2**136` after the
    ↪ first branch above.
    z := shr(18, mul(z, add(shr(r, x), 65536))) // A `mul()` is saved
    ↪ from starting `z` at 181.
  }
}

```

- Step 6: We can now test the two core sqrt functions for equivalence using Halmos.

```

// halmos returns and error "counterexample-unknown". So we disable the
↪ timeout limit for the assertion; thus giving Halmos more time to
↪ find a counterexample.
/// @custom:halmos --solver-timeout-assertion 0

function check_coreSqrtFunctions() public {
  // ARRANGE: specify input conditions
  uint256 x = svm.createUint256("x");

```

```

    // ACT: call target contracts
    uint256 coreMathMastersOuput = Base_Test.coreMathMasterSqrt(x);
    uint256 coreSoladyOutput = Base_Test.coreSoladySqrt(x);

    // ASSERT: check output states
    assertEq(coreMathMastersOuput, coreSoladyOutput);
}

```

- Step 7: Having received a counter example from Halmos, let's test it using Foundry's Forge tool. Run it using "forge test -match-test test_poc_sqrt -vv"

```

function test_poc_sqrt() public {
    // ARRANGE: specify input conditions
    uint256 x =
↪ 105311293498665291426722909308999732236070323463302251608708546560;

    // ACT: call target contracts
    uint256 mathMastersOuput = Base_Test.coreMathMasterSqrt(x);
    uint256 soladyOutput = Base_Test.coreSoladySqrt(x);
    console.log("mathMastersOuput: ", mathMastersOuput);
    console.log("soladyOutput: ", soladyOutput);

    // ASSERT: check output states
    assertEq(mathMastersOuput, soladyOutput);
}

```

Impact

The loss of precision can lead to incorrect results when using the MathMasters implementation.

Tools Used

- Foundry Forge: Fuzz Testing for Correctness
- Halmos: Formal Verification Testing for Equivalence

Recommendations

Use hexadecimal to represent the constants in the MathMasters implementation.

```

- let r := shl(7, lt(87112285931760246646623899502532662132735, x))
- r := or(r, shl(6, lt(4722366482869645213695, shr(r, x))))
- r := or(r, shl(5, lt(1099511627775, shr(r, x))))
- r := or(r, shl(4, lt(16777002, shr(r, x))))

+ let r := shl(7, lt(0xffffffffffffffffffffffffffffffff, x))

```

```
+ r := or(r, shl(6, lt(0xffffffffffffffff, shr(r, x))))
+ r := or(r, shl(5, lt(0xffffffff, shr(r, x))))
+ r := or(r, shl(4, lt(0xffff, shr(r, x))))
```

Low

[L-1] Version compatibility issue prevents use of library for contracts using version 0.8.3 of Solidity

Relevant GitHub Links

<https://github.com/Cyfrin/2024-01-math-master/blob/main/src/MathMasters.sol#L3>

Summary

There are compatibility issues between the library and smart contracts using version 0.8.3 of Solidity.

Vulnerability Details

Custom errors were introduced in Solidity version 0.8.4. This prevents smart contracts using version 0.8.3 from using this library.

References: Solidity 0.8.4 Release Announcement

Impact

Smart contracts using version 0.8.3 of Solidity cannot use this library.

Proof of Concept (PoC)

If we attempt to compile a smart contract that uses version 0.8.3 of Solidity and includes the library, a compilation error will occur.

Let's create a new smart contract in `src/MathMastersExposed.sol` with a version 0.8.3 of Solidity that will include the library:

```
// SPDX-License-Identifier: AGPL-3.0-only
pragma solidity 0.8.3;
```

```
import {MathMasters} from "src/MathMasters.sol";
```

```
contract MathMastersExposed {
    using MathMasters for uint256;
}
```


If we attempt to compile this smart contract using `forge build`, we will encounter a compilation error:

Compiler run failed:

```
Error (2314): Expected ';' but got '('
--> src/CustomErrors.sol:5:36:
```

```
5 |      error MathMasters__MulWadFailed(
  |                                     ^
```

```
Error (2314): Expected ';' but got '('
--> src/MathMasters.sol:14:41:
```

```
14 |      error MathMasters__FactorialOverflow(
  |
```

Tools Used

- Manual review
- Foundry

Recommendations

The library's pragma should not include version 0.8.3.

Recommended changes to the `MathMasters.sol` library:

```
- pragma solidity ^0.8.3;
+ pragma solidity ^0.8.4;
```

If we change the pragma from 0.8.3 to 0.8.4 in our previously created smart contract, we can now compile it successfully.

[L-2] In `MathMasters::mulWad` and `MathMasters::mulWadUp` functions the revert reason is empty

Relevant GitHub Links

<https://github.com/Cyfrin/2024-01-math-master/blob/84c149baf09c1558d7ba3493c7c4e68d83e7b3aa/src/MathMasters.sol#L41>

<https://github.com/Cyfrin/2024-01-math-master/blob/84c149baf09c1558d7ba3493c7c4e68d83e7b3aa/src/MathMasters.sol#L54>

Summary

In the `MathMasters::mulWad` and `MathMasters::mulWadUp` functions the error selector is wrong and the `revert` read from empty slot.

Vulnerability Details

In the `MathMasters::mulWad` and `MathMasters::mulWadUp` is used the error selector `0xbac65e5b` for the cases when the functions revert.

```
function mulWad(uint256 x, uint256 y) internal pure returns (uint256 z) {
    // @solidity memory-safe-assembly
    assembly {
        // Equivalent to `require(y == 0 || x <= type(uint256).max / y)`.
        if mul(y, gt(x, div(not(0), y))) {
@>         mstore(0x40, 0xbac65e5b) // `MathMasters__MulWadFailed()`.
@>         revert(0x1c, 0x04)
        }
        z := div(mul(x, y), WAD)
    }
}

function mulWadUp(uint256 x, uint256 y) internal pure returns (uint256 z) {
    /// @solidity memory-safe-assembly
    assembly {
        // Equivalent to `require(y == 0 || x <= type(uint256).max / y)`.
        if mul(y, gt(x, or(div(not(0), y), x))) {
@>         mstore(0x40, 0xbac65e5b) // `MathMasters__MulWadFailed()`.
@>         revert(0x1c, 0x04)
        }
        if iszero(sub(div(add(z, x), y), 1)) { x := add(x, 1) }
        z := add(iszero(iszero(mod(mul(x, y), WAD))), div(mul(x, y), WAD))
    }
}
```

But `0xbac65e5b` is a selector for `MulWadFailed` error from `SoLadY` library not for a `MathMasters__MulWadFailed` error from `MathMasters` contract. The right selector for the error `MathMasters__MulWadFailed` from `MathMasters` contract is `0xa56044f7`. This can be retrieved by using `chisel` and the command: `cast sig "MathMasters__MulWadFailed()"`.

Additionally, the `revert` error is written to `0x40` and after that is read in the `revert` from `0x1c`. That is incorrect and leads to empty reason for `revert`.

Impact

The functions `MathMasters::mulWad` and `MathMasters::mulWadUp` use wrong error selec-

tor and by reverting the reason is not `MathMasters__MulWadFailed`.

Also, the `mstore(0x40, 0xbac65e5b)` statement stores the error message identifier at memory location `0x40`, but the `revert(0x1c, 0x04)` statement is trying to read from memory location `0x1c`. These are different memory locations. The `revert(0x1c, 0x04)` statement will read the data stored at memory location `0x1c`, which in this case is undefined (it is empty) since it hasn't been set anywhere else in the function. But if we change the memory location, the error message will be `custom error 0xbac65e5b`, because the selector of this error is not defined in this contract. It is from the `SoLady` library.

Tools Used

VS Code, Foundry

Recommendations

Change `mstore(0x40, 0xbac65e5b)` to `mstore(0x00, 0xa56044f7)` in `MathMasters::mulWad` and `MathMasters::mulWadUp` functions to have Reason: `MathMasters__MulWadFailed()` by reverting:

```
function mulWad(uint256 x, uint256 y) internal pure returns (uint256 z) {
    // @solidity memory-safe-assembly
    assembly {
        // Equivalent to `require(y == 0 || x <= type(uint256).max / y)`.
        if mul(y, gt(x, div(not(0), y))) {
+           mstore(0x00, 0xa56044f7)
-           mstore(0x40, 0xbac65e5b) // `MathMasters__MulWadFailed()`.
            revert(0x1c, 0x04)
        }
        z := div(mul(x, y), WAD)
    }
}
```

```
function mulWadUp(uint256 x, uint256 y) internal pure returns (uint256 z) {
    /// @solidity memory-safe-assembly
    assembly {
        // Equivalent to `require(y == 0 || x <= type(uint256).max / y)`.
        if mul(y, gt(x, or(div(not(0), y), x))) {
+           mstore(0x00, 0xa56044f7)
-           mstore(0x40, 0xbac65e5b) // `MathMasters__MulWadFailed()`.
            revert(0x1c, 0x04)
        }
        if iszero(sub(div(add(z, x), y), 1)) { x := add(x, 1) }
        z := add(iszero(iszero(mod(mul(x, y), WAD))), div(mul(x, y), WAD))
    }
}
```

```
}  
}
```

[L-3] In MathMasters::mulWad and MathMasters::mulWadUp functions the revert reason is empty

Relevant GitHub Links

<https://github.com/Cyfrin/2024-01-math-master/blob/84c149baf09c1558d7ba3493c7c4e68d83e7b3aa/src/MathMasters.sol#L41>

<https://github.com/Cyfrin/2024-01-math-master/blob/84c149baf09c1558d7ba3493c7c4e68d83e7b3aa/src/MathMasters.sol#L54>

Summary

In the MathMasters::mulWad and MathMasters::mulWadUp functions the error selector is wrong and the revert read from empty slot.

Vulnerability Details

In the MathMasters::mulWad and MathMasters::mulWadUp is used the error selector 0xbac65e5b for the cases when the functions revert.

```
function mulWad(uint256 x, uint256 y) internal pure returns (uint256 z) {  
    // @solidity memory-safe-assembly  
    assembly {  
        // Equivalent to `require(y == 0 || x <= type(uint256).max / y)`.  
        if mul(y, gt(x, div(not(0), y))) {  
@>            mstore(0x40, 0xbac65e5b) // `MathMasters__MulWadFailed()`.  
@>            revert(0x1c, 0x04)  
        }  
        z := div(mul(x, y), WAD)  
    }  
}
```

```
function mulWadUp(uint256 x, uint256 y) internal pure returns (uint256 z) {  
    /// @solidity memory-safe-assembly  
    assembly {  
        // Equivalent to `require(y == 0 || x <= type(uint256).max / y)`.  
        if mul(y, gt(x, or(div(not(0), y), x))) {  
@>            mstore(0x40, 0xbac65e5b) // `MathMasters__MulWadFailed()`.  
@>            revert(0x1c, 0x04)  
        }  
    }  
}
```

```

    if iszero(sub(div(add(z, x), y), 1)) { x := add(x, 1) }
    z := add(iszero(iszero(mod(mul(x, y), WAD))), div(mul(x, y), WAD))
  }
}

```

But 0xbac65e5b is a selector for MulWadFailed error from Solady library not for a MathMasters__MulWadFailed error from MathMasters contract. The right selector for the error MathMasters__MulWadFailed from MathMasters contract is 0xa56044f7. This can be retrieved by using chisel and the command: `cast sig "MathMasters__MulWadFailed()"`.

Additionally, the revert error is written to 0x40 and after that is read in the revert from 0x1c. That is incorrect and leads to empty reason for revert.

Impact

The functions `MathMasters::mulWad` and `MathMasters::mulWadUp` use wrong error selector and by reverting the reason is not `MathMasters__MulWadFailed`.

Also, the `mstore(0x40, 0xbac65e5b)` statement stores the error message identifier at memory location 0x40, but the `revert(0x1c, 0x04)` statement is trying to read from memory location 0x1c. These are different memory locations. The `revert(0x1c, 0x04)` statement will read the data stored at memory location 0x1c, which in this case is undefined (it is empty) since it hasn't been set anywhere else in the function. But if we change the memory location, the error message will be custom error 0xbac65e5b, because the selector of this error is not defined in this contract. It is from the Solady library.

Tools Used

VS Code, Foundry

Recommendations

Change `mstore(0x40, 0xbac65e5b)` to `mstore(0x00, 0xa56044f7)` in `MathMasters::mulWad` and `MathMasters::mulWadUp` functions to have Reason: `MathMasters__MulWadFailed()` by reverting:

```

function mulWad(uint256 x, uint256 y) internal pure returns (uint256 z) {
  // @solidity memory-safe-assembly
  assembly {
    // Equivalent to `require(y == 0 || x <= type(uint256).max / y)`.
    if mul(y, gt(x, div(not(0), y))) {
+     mstore(0x00, 0xa56044f7)
-     mstore(0x40, 0xbac65e5b) // `MathMasters__MulWadFailed()`.
    revert(0x1c, 0x04)
  }
}

```

```
    }
    z := div(mul(x, y), WAD)
  }
}

function mulWadUp(uint256 x, uint256 y) internal pure returns (uint256 z) {
  /// @solidity memory-safe-assembly
  assembly {
    // Equivalent to `require(y == 0 || x <= type(uint256).max / y)`.
    if mul(y, gt(x, or(div(not(0), y), x))) {
+      mstore(0x00, 0xa56044f7)
-      mstore(0x40, 0xbac65e5b) // `MathMasters__MulWadFailed()`.
      revert(0x1c, 0x04)
    }
    if iszero(sub(div(add(z, x), y), 1)) { x := add(x, 1) }
    z := add(iszero(iszero(mod(mul(x, y), WAD))), div(mul(x, y), WAD))
  }
}
```